

Machine Learning Operations AIMLCZG523 - Assignment 01

Heart Disease Risk Prediction using Raw UCI 76-Attribute Data Files Only

Prepared as a production-style MLOps solution covering raw data acquisition, EDA, feature engineering, model training, MLflow tracking, API deployment, CI/CD, Docker, Kubernetes, and monitoring.

1. Project Overview

The project develops a reproducible machine learning service that predicts heart disease risk from patient health attributes. This version follows a strict raw-only data policy: the training dataset is created only from the original unprocessed UCI 76-attribute files. The processed UCI files are deliberately not used for model development.

Item	Project decision
Dataset source	Official UCI Heart Disease raw 76-attribute files
Raw training files	cleveland.data, hungarian.data, switzerland.data, long-beach-va.data
Processed files	Not used for training in this raw-only version
Target conversion	Original num: 0 = no disease, 1-4 = heart disease risk present
Deployment interface	FastAPI JSON prediction service

2. Raw-Only Data Acquisition

The data acquisition script downloads official UCI metadata, raw data files, and the Costs folder for traceability. The model-ready dataset is built by tokenizing each raw file, chunking records into 76 attributes, and extracting the 14 documented ML variables.

```
python src/data_acquisition.py
```

Raw file	Use in project
cleveland.data	Used for training after extracting selected attributes
hungarian.data	Used for training after extracting selected attributes
switzerland.data	Used for training after extracting selected attributes
long-beach-va.data	Used for training after extracting selected attributes
new.data	Downloaded for traceability, not used for training
Costs/*	Downloaded for completeness, not patient training data

3. Feature Set and Target Definition

The raw files contain 76 attributes. The pipeline extracts the 14 variables commonly used for ML experiments: 13 predictors and the diagnosis target. Missing values such as -9, -9.0, and ? are converted to nulls and handled inside the sklearn preprocessing pipeline.

Feature	Description
age	Age in years
sex	Patient sex
cp	Chest pain type
trestbps	Resting blood pressure
chol	Serum cholesterol
fbs	Fasting blood sugar flag
restecg	Resting ECG result
thalach	Maximum heart rate
exang	Exercise-induced angina
oldpeak	ST depression

Feature	Description
slope	ST segment slope
ca	Major vessels colored by fluoroscopy
thal	Thalassemia result
target	Binary diagnosis label

4. EDA and Data Quality

EDA is generated after running the raw-only acquisition script. The notebook and training script produce class balance, histograms, correlation heatmap, confusion matrix, and ROC curves. Screenshots must be captured from actual execution and placed under the screenshots folder.

EDA item	Evidence file/location
Class balance	artifacts/figures/eda_class_balance.png
Feature histograms	artifacts/figures/eda_histograms.png
Correlation heatmap	artifacts/figures/eda_correlation_heatmap.png
Missing-value profile	data/processed/heart_disease_raw_only_all_sites_cleaned.profile.json

5. Preprocessing and Modeling

The training pipeline uses a ColumnTransformer to apply median imputation and scaling for numerical variables and most-frequent imputation plus one-hot encoding for categorical variables. The same fitted pipeline is stored with the selected model to guarantee consistent inference behavior.

Component	Implementation
Numerical pipeline	SimpleImputer(strategy=median) + StandardScaler
Categorical pipeline	SimpleImputer(strategy=most_frequent) + OneHotEncoder(handle_unknown=ignore)
Models	Logistic Regression, Random Forest, Support Vector Machine
Validation	Stratified train-test split and cross-validation through GridSearchCV
Selection priority	Recall, then ROC-AUC, then F1 due to healthcare-style false-negative risk

6. Experiment Tracking with MLflow

MLflow records model type, hyperparameters, cross-validation score, test metrics, confusion matrix, ROC curve, classification report, and the trained model artifact. The experiment name is raw-only specific: heart-disease-risk-prediction-mlops-raw-76-only.

```
mlflow ui --backend-store-uri mlruns --host 127.0.0.1 --port 5000
```

7. API Design

The deployed service is implemented with FastAPI. The API validates input using Pydantic, loads the stored sklearn pipeline, returns the predicted class, a readable risk label, confidence, probability, model name, and model version.

Endpoint	Purpose
GET /	Service landing message
GET /health	Model artifact and API health check
POST /predict	Patient JSON prediction
GET /metrics	Prometheus metrics

8. Docker, CI/CD, and Deployment

Docker packages the API, source code, dependencies, raw-only acquisition, and training workflow. GitHub Actions installs dependencies, runs linting, runs tests, validates training, and builds Docker. Deployment can be demonstrated using Render/Railway or local Kubernetes through Minikube or Docker Desktop Kubernetes.

```
docker build -t heart-disease-mlops:latest .
docker run --rm -p 8000:8000 heart-disease-mlops:latest
```

9. Monitoring and Logging

The FastAPI application logs every request with method, endpoint, status, and latency. Prometheus counters and histograms track total requests, predictions, errors, and request latency. This satisfies the basic production monitoring requirement for uptime, request volume, error count, latency, and prediction count.

10. Limitations and Future Improvements

The raw files contain missing values and historical medical data, so the model must not be used for real clinical decisions. Future improvements include adding drift monitoring, model registry approval gates, SHAP explanations, stronger validation, secure deployment, and a retraining workflow. Public deployment evidence and screenshots must be generated from real execution.

11. Reproducible Execution Commands

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python src/data_acquisition.py
python src/train.py
pytest -q
python -m uvicorn api.main:app --host 0.0.0.0 --port 8000
```

12. Conclusion

This raw-only version provides a stronger academic evidence trail because the model-ready dataset is generated directly from the original UCI raw files rather than relying on processed copies. The solution remains production-oriented through reusable code, automated testing, experiment tracking, API serving, containerization, deployment manifests, and monitoring assets.